

Code -

```
#include <iostream>
#include <vector>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void parallelDFSUtil(int node, vector<bool> &visited) {
        bool alreadyVisited;

        #pragma omp critical
        {
            alreadyVisited = visited[node];
            if (!visited[node]) {
                visited[node] = true;
                cout << node << " ";
            }
        }

        if (alreadyVisited) return;

        #pragma omp parallel for
        for (int i = 0; i < adj[node].size(); i++) {
            int neighbor = adj[node][i];

            if (!visited[neighbor]) {
                #pragma omp task
                parallelDFSUtil(neighbor, visited);
            }
        }
    }

    void parallelDFS(int start) {
        vector<bool> visited(V, false);

        cout << "\nParallel DFS Traversal: ";

        #pragma omp parallel
        {
            #pragma omp single
            {
                parallelDFSUtil(start, visited);
            }
        }

        cout << endl;
    }
};
```

```

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    g.parallelDFS(start);

    return 0;
}

```

Output -

```

Enter number of vertices: 6
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 5
Enter starting vertex: 0

Parallel DFS Traversal: 0 1 3 4 2 5

```

=== Code Execution Successful ===